

ISSUE NO. 02

GraphRAG: Knowledge Graphs Meet Vector DBs

Mapping multi-hop entity relationships that vector search alone can't detect

By Sai / Full-Stack x AI Engineering / 6 min read

Welcome to Issue #2 of The Full-Stack AI Playbook!

In Issue #1, we tore down Naive RAG and established the production triple-threat: parent-child chunking, hybrid search (dense + BM25), and cross-encoder reranking. But even with precise vector search and reranking, pure vector databases hit an invisible wall when your queries require multi-hop reasoning or global entity understanding.

If you ask your system: **"How does Customer X's compliance policy impact Service Y's database migration?"** — vector similarity fails because those pieces of context live in separate documents with zero semantic overlap.

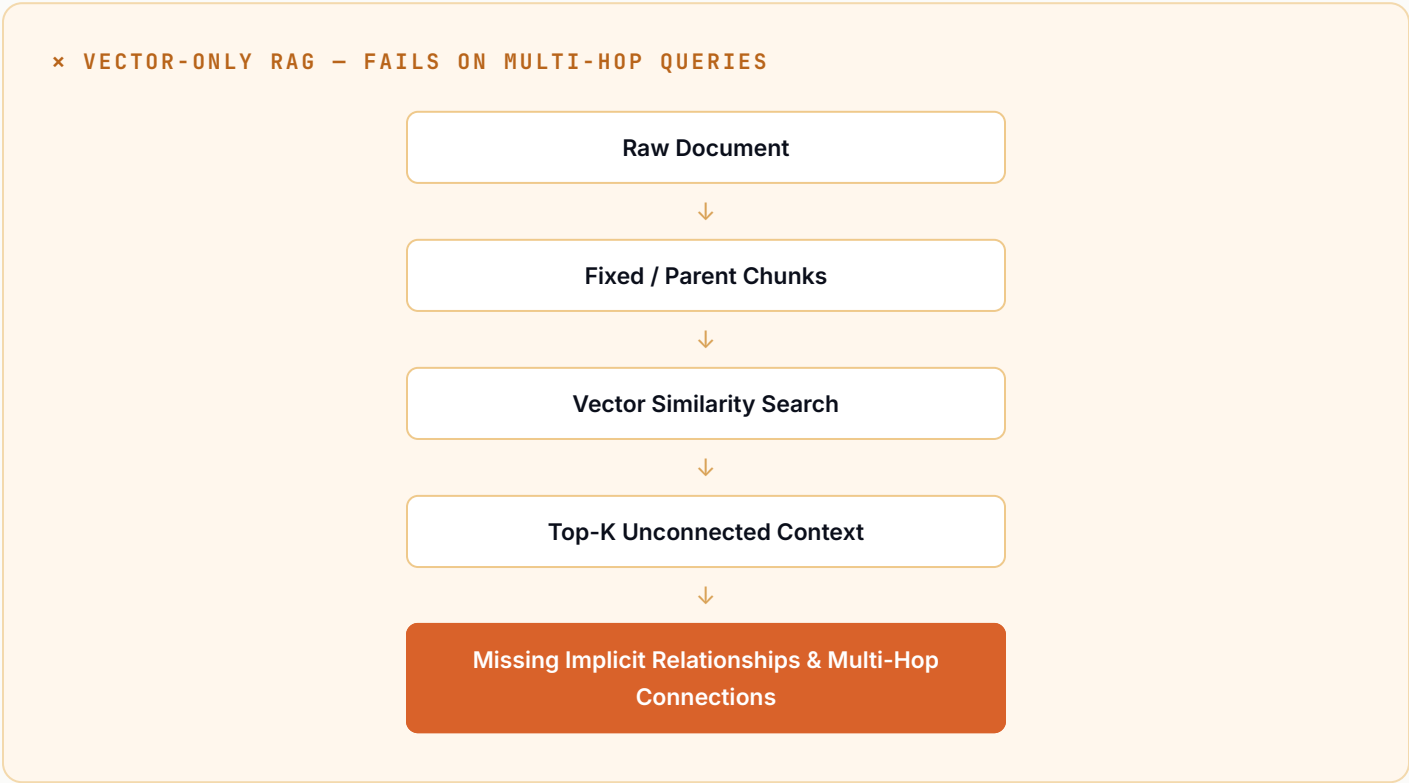
This issue breaks down **GraphRAG**: combining Knowledge Graphs (Neo4j) with vector databases to map multi-hop entity relationships that vector search alone can never detect.

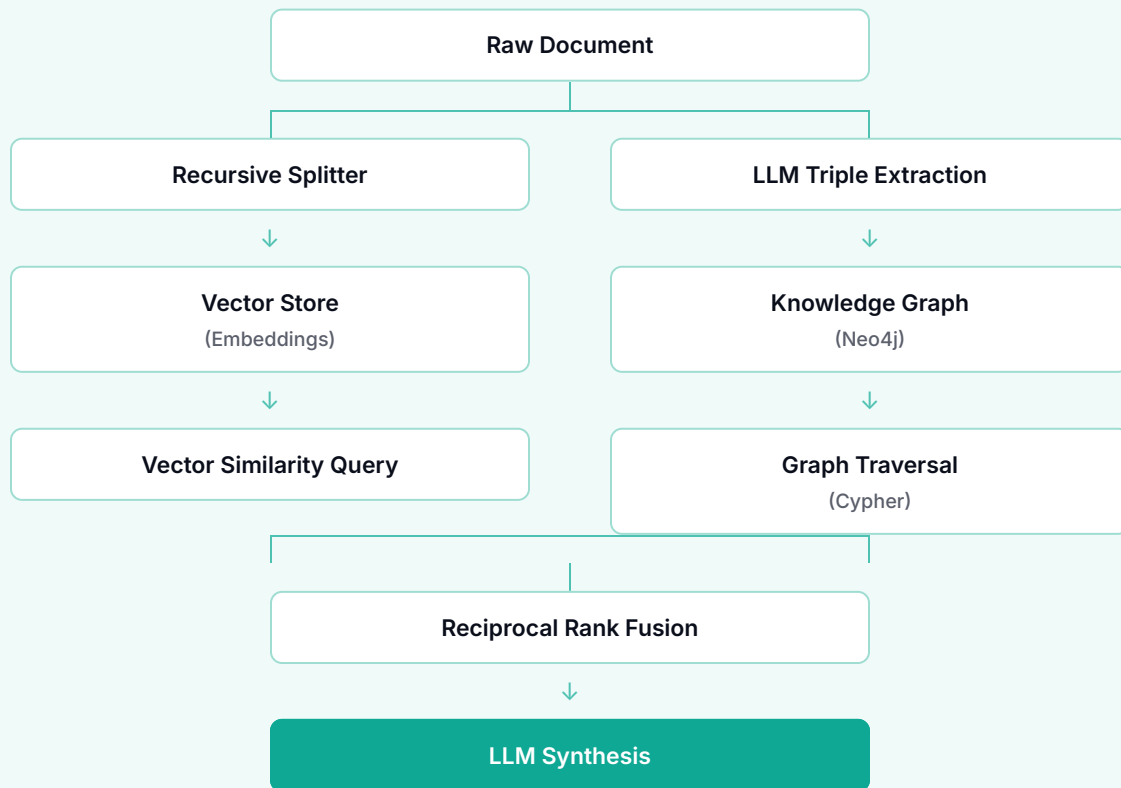
WHAT YOU'LL WALK AWAY WITH

- ✓ Why vector search fails on multi-hop entity queries
- ✓ Knowledge graph extraction & triple generation
- ✓ The hybrid GraphRAG architecture pattern
- ✓ A working Python snippet for graph + vector querying
- ✓ Latency, cost, and complexity trade-offs

The Vector vs. GraphRAG Retrieval Pipeline

Here's the structural difference between vector-only retrieval and a hybrid GraphRAG pipeline.





§ UPGRADES 3 Architectural Upgrades You Need for GraphRAG

1 Entity & Triple Extraction

THE PROBLEM

Vector databases treat chunks as isolated islands of text. They have no concept of "Entity A is owned by Entity B" unless both are explicitly typed in the same 400-token window.

THE FIX

Pass documents through an extraction pipeline that identifies entities (nodes) and relationships (edges), formatted as triples: `(Service_A) -[:DEPENDS_ON]→ (Database_B)`. Store these nodes and edges inside a knowledge graph like Neo4j.

2

Cypher Graph Traversal + Vector Hybrid Querying

THE PROBLEM

Graph databases are incredible at explicit pathfinding (`MATCH (a)-[:REL]→(b)`), but terrible at broad semantic fuzzy matching.

THE FIX

Run hybrid retrieval. Use vector embeddings to identify the initial entry-point nodes in the graph (semantic matching), then execute 1-hop or 2-hop Cypher queries to pull all structurally connected context.

3

Graph Community Summarization

THE PROBLEM

High-level summary queries like "What are the main architectural risks across the entire repository?" fail in standard RAG because no single top-k retrieval covers the whole dataset.

THE FIX

Group your knowledge graph into hierarchical communities (using algorithms like Leiden/Louvain), pre-summarize each cluster with an LLM, and query those high-level community summaries at runtime.

Code: Wiring Up a Hybrid GraphRAG

Retriever

A clean Python implementation pattern demonstrating vector seed retrieval combined with Neo4j graph traversal.

graphrag_retriever.py

```
import os
from langchain_community.graphs import Neo4jGraph
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma

# 1. Initialize Vector Store & Knowledge Graph
graph = Neo4jGraph(
    url=os.getenv("NEO4J_URI"),
    username=os.getenv("NEO4J_USER"),
    password=os.getenv("NEO4J_PASSWORD")
)

vectorstore = Chroma(
    collection_name="graph_seeds",
    embedding_function=OpenAIEmbeddings(model="text-embedding-3-small")
)

# 2. Hybrid Graph Retrieval Function
def retrieve_graph_context(query: str) -> str:
    # Step A: Perform vector search to find anchor entity names
    initial_docs = vectorstore.similarity_search(query, k=2)
    entity_names = [doc.metadata["entity_name"] for doc in initial_docs]

    # Step B: Run 2-Hop Cypher Graph Traversal in Neo4j
    cypher_query = """
    MATCH (e:Entity)-[r:RELATION*1..2]-(connected:Entity)
    WHERE e.name IN $entities
    RETURN e.name + ' ' + type(r[0]) + ' ' + connected.name AS relationship
    LIMIT 15
    """

    graph_results = graph.query(cypher_query, params={"entities": entity_names})

    # Step C: Format Graph Triples for LLM Prompt Context
    relationships = [item["relationship"] for item in graph_results]
    return "\n".join(relationships)

# Example Usage
context = retrieve_graph_context("How does Database_B failure impact Service_A?")
```

Trade-Off Breakdown

Before pushing GraphRAG to production, keep these operational metrics in mind.

STRATEGY	LATENCY IMPACT	COST IMPACT	ACCURACY GAIN
Naive RAG	Baseline (~200ms)	Low	HIGH HALLUCINATION • LOW ACCURACY
Production Vector RAG	+140–220ms	Low–Medium	

GraphRAG (Hybrid)

+250–400ms

Medium-High
(graph extraction)

EXTREME · SOLVES MULTI-HOP

WHAT'S NEXT — ISSUE #3

Autonomous AI Agents in Production: From Loops to Finite State Machines

Moving from static retrieval pipelines to autonomous execution loops. Next issue, we cover how to build production-grade AI agents using **LangGraph** to eliminate infinite loops, handle tool failures gracefully, and maintain state across multi-step tasks.

What's the hardest multi-hop retrieval query your team has tried to solve? Drop a comment or reply directly to this issue — I read every response.

Until next time, keep building.

— Sai

New issue every two weeks — repo setups, architecture diagrams, benchmarks, and real production post-mortems for full-stack AI systems.

[SUBSCRIBE →](#)

#FullStackAIPlaybook #SoftwareEngineering #SystemDesign #ArtificialIntelligence #BuildWithSai